

Abstractive Text Summarisation using Pegasus

Documentation and Assignment

1 Introduction

Text summarisation is the task of condensing a long piece of text into a shorter version while preserving its core meaning. There are two broad approaches:

- **Extractive Summarisation** — selects and stitches together important sentences/phrases directly from the source text.
- **Abstractive Summarisation** — generates new sentences that capture the meaning of the source text, similar to how a human would paraphrase and summarise. It can produce words/phrases not present in the original text.

Pegasus (Pre-training with Extracted Gap-sentences for Abstractive Summarization) is a Transformer-based model developed by Google Research, specifically designed and pre-trained for abstractive summarisation tasks.

2 Architecture Overview

Pegasus is built on the standard **Transformer encoder-decoder** architecture (similar to the original “Attention is All You Need” model), with a specialised pre-training objective.

Component	Role
Encoder	Reads and encodes the input document into contextual representations
Decoder	Generates the summary token-by-token, attending to encoder outputs
Self-Attention	Captures relationships between words/sentences across the document
Cross-Attention	Allows decoder to focus on relevant parts of the input while generating

2.1 Key Innovation: Gap Sentence Generation (GSG)

Unlike BERT-style masked language modelling, Pegasus is pre-trained using a novel objective called **Gap Sentence Generation (GSG)** and **Masked Language Modelling (MLM)**:

1. Important sentences are identified in a document (using heuristics like ROUGE overlap with the rest of the document).
2. These “important” sentences are masked/removed from the input.
3. The model is trained to generate these masked sentences as output — mimicking the actual summarisation task during pre-training.
4. In MLM, 15% of the tokens in the input text are selected following BERT. The selected tokens are replaced in one of three ways: 80% of the time by a mask token [MASK2], 10% of the time by a random token, or 10% of the time without any change

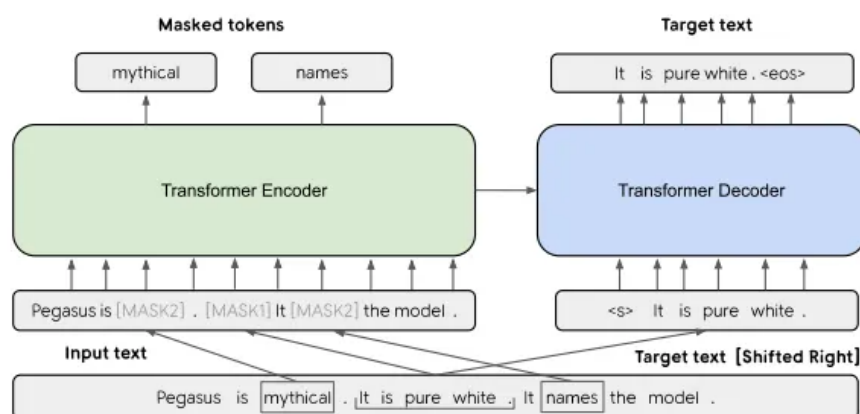


Figure 1: The Encode Decoder architecture behind Pegasus. The input tokens are masked based on GSG and MLM methods and output metrics (ROUGE Scores) are calculated to evaluate model’s performance.

This makes Pegasus’s pre-training objective closely resemble the downstream fine-tuning task (summarisation), which is why it performs strongly even with limited fine-tuning data.

3 Why Pegasus for Abstractive Summarisation?

- Pre-training objective is specifically aligned with summarisation (not generic language modelling).
- Achieves strong performance on benchmark datasets (CNN/DailyMail, XSum, Reddit TIFU, arXiv, PubMed, etc.).
- Performs well even in **low-resource** settings (fine-tuning with as few as 1,000 examples).
- Available in multiple pre-trained checkpoints for different domains (news, scientific papers, etc.) via Hugging Face.

4 General steps

- Install the dependencies. You can view the attached links for the list of modules.
- Understand what is happening behind the scene.
- Load the model.
- Summarise the input text using the model

5 Evaluating Summaries — ROUGE Score

Summaries are typically evaluated using **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**, which measures overlap between generated and reference summaries.

```
1 pip install rouge-score
```

- **ROUGE-1:** Unigram overlap
- **ROUGE-2:** Bigram overlap

- **ROUGE-L**: Longest common subsequence overlap

6 Assignment

Objective

Apply the Pegasus model to perform abstractive summarisation and evaluate its performance.

Part A — Setup & Basic Summarisation (Beginner)

1. Install the required libraries and load the `google/pegasus-xsum` checkpoint.
2. Pick any 3 news articles (at least 200 words each) and generate summaries using Pegasus.
3. Report the original word count and summary word count for each article.

Part B — Parameter Exploration (Intermediate)

4. For one article, generate summaries using three different values of `num_beams` (e.g., 1, 4, 8) and compare the outputs.
5. Repeat with different `length_penalty` values (e.g., 0.8, 1.0, 2.0). Discuss how each parameter affects summary quality and length.

Part C — Evaluation (Intermediate)

6. For each article in Part A, manually write a reference (human) summary.
7. Compute ROUGE-1, ROUGE-2, and ROUGE-L scores comparing Pegasus's output to your reference summaries.
8. Present your scores in a table and briefly interpret the results.

Part D — Comparative Analysis (Advanced)

9. Compare Pegasus's output with an extractive method (e.g., using `sumy` or `TextRank`) on the same articles.
10. Write a short reflection covering:
 - Differences you observed between abstractive and extractive summaries
 - Any factual inconsistencies/hallucinations you noticed in Pegasus output
 - Situations where you'd prefer extractive vs abstractive summarisation

Part E (Optional, Bonus) — Domain Checkpoint Comparison

11. Try summarising a scientific abstract using both `pegasus-xsum` and `pegasus-pubmed` (or `pegasus-arxiv`). Compare which checkpoint produces more coherent, domain-appropriate summaries.

For domain-specific summarisation, Pegasus can be fine-tuned on custom datasets using Hugging Face's `Seq2SeqTrainer`:

```
1 from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments
2
3 training_args = Seq2SeqTrainingArguments(
4     output_dir="./pegasus-finetuned",
```

```
5     per_device_train_batch_size=2,
6     per_device_eval_batch_size=2,
7     num_train_epochs=3,
8     predict_with_generate=True,
9     evaluation_strategy="epoch",
10    save_strategy="epoch",
11 )
12
13 trainer = Seq2SeqTrainer(
14     model=model,
15     args=training_args,
16     train_dataset=train_dataset,    # pre-tokenized dataset
17     eval_dataset=eval_dataset,
18 )
19
20 trainer.train()
```

7 Common Challenges & Tips

- **Repetition/hallucination:** Use `no_repeat_ngram_size=3` and moderate `length_penalty`.
- **Long documents:** Pegasus has an input token limit (typically 512 or 1024 tokens depending on checkpoint) — long documents need chunking or truncation.
- **Domain mismatch:** Choose the checkpoint closest to your domain (`pegasus-cnn_dailymail` for news, `pegasus-pubmed` for medical text, `pegasus-arxiv` for scientific papers).
- **Evaluation limits:** ROUGE measures lexical overlap, not factual correctness — always spot-check generated summaries manually.

Deliverables

- A Jupyter notebook / Python script with all code and outputs
- A short report (PDF or Markdown) containing your ROUGE tables, comparisons, and written reflection

8 References for Further Reading

- Zhang, J. et al. “PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization” (ICML 2020)— [Paper link](#)
- Hugging Face Transformers documentation — [Pegasus model card](#)